

LAYER 1 — CLIENTS

Public Web (Patient)

React + Vite SPA
Pages:
• Lease Request Form
• Token Status Lookup
No login required

Staff Web (Internal)

React + Vite SPA
Admin:
Centers, Users, SKUs, Assets, Reports
Approver:
Request queue, Approve / Reject
Center Admin:
Inventory, Issue Device, Return Device, Transfer
Asset Manager:
Onboarding, Transfer Initiation
Login via OTP (mobile)

Monitoring / Health

Deployment health probe
GET /health

! HTTPS - JSON REST - no auth headers (Public)
! HTTPS - JSON REST - X-Actor-Mobile + X-Actor-Role headers (Staff)
! GET /health (Monitoring)

LAYER 2 — FastAPI Backend (Python) v0.2.0

Auth: X-Actor-Mobile + X-Actor-Role headers (no JWT) · CORS: localhost:8080 (configurable via CORS_ORIGINS env var)

MIDDLEWARE: CORS · Status Translation (API→DB) · Audit Log Writer · JSON Serializer (datetime, UUID, Decimal)

Health

GET /health

Centers

GET /api/v1/centers
POST /api/v1/centers

Users

GET /api/v1/users
POST /api/v1/users
PATCH /api/v1/users/{id}
DELETE /api/v1/users/{id}

SKUs & Assets

GET/POST /api/v1/skus
PATCH /api/v1/skus/{id}
GET/POST /api/v1/assets
PATCH /api/v1/assets/{id}

Asset Transfers

GET /api/v1/asset-transfers
POST /api/v1/asset-transfers
PATCH /api/v1/asset-transfers/{id}

Lease Requests

GET/POST /api/v1/lease-requests
PATCH /api/v1/lease-requests/{id}
GET /api/v1/lease-requests/by-token/{token}
GET .../by-token/{token}/issue-context/{center_id}
Issue-context: returns local stock count + assets/centers list

Audit Logs

GET /api/v1/audit-logs
Filters: actions, token, start_at, end_at, limit, offset

Business Logic

* resolve_lease_db_status() → server-asynchronous lease fulfillment. Frontend cannot force a lease status.
* asset_status_db(api) → under_repair → repair translation
* lease_status_db(api) → active → issued, closed → returned
* _action_for_tag() → routes audit rows: admin | approver | requests | centers
* _actor_context() → X-Actor-Mobile header → user DB lookup
! Audit writes on separate commit → no single-transaction guarantee
! No server-side role enforcement on any endpoint

! SQLAlchemy ORM - pympd driver - connection pool - pool_pre_ping=True

LAYER 3 — MySQL Database (mahesh_meds)

Charset: utf8mb4 · All PKs: CHAR(36) UUID · All PKs generated in Python (not DB-side)

Master Data

centers

id (PK UUID), name, city, state, is_active
contact_person, contact_phone, pin_code

users

id (PK UUID), name, role (ENUM)
role ENUM: admin | center_admin | approver | asset_manager
center_id (FK → centers), is_active
Role stored here but NOT enforced by backend

Authentication

otp_sessions

id (PK UUID), sku_id, otp_code
expires_at, is_used
attempts → brute-force protection

Catalogue & Inventory

skus

id (PK UUID), sku_code (UNIQUE), name
category, description, image_url, is_active

assets

id (PK UUID), sku_id (FK → skus)
serial_number (UNIQUE), center_id (FK → centers)
status ENUM: available | leased | repair | retired | in Transit
purchase_date, warranty_expiry, invoice_number, qr_code

Lease Management

lease_requests

id (PK UUID), token_number (UNIQUE)
requestor_name, mobile, address_number
preferred_center_id (FK → centers)
status ENUM: pending | rejected | issued | returned | partially_returned
approved_by (FK → users) → populated at approval transition
approval_comments, rejection_reason

lease_items — THE BRIDGE TABLE

id (PK UUID)
lease_request_id (FK → lease_requests, CASCADE delete)
sku_id (FK → skus) → INTENT: set at creation, never changes
asset_id (FK → assets, NULLABLE → FULFILLMENT)
NULL = approved but not yet issued
Filled = triggers issued status via resolve_lease_db_status()
quantity_requested, approved_quantity
issued_at, issued_by (FK → users)
due_date, returned_at
returned_to (FK → users)
return_center_id (FK → centers) → may differ from issuing center
condition_on_return

Operations

asset_transfers

id (PK UUID), asset_id (FK → assets)
from_center_id (FK → centers), to_center_id (FK → centers)
transfer_reason ENUM: rebalancing | repair | request | other
initiated_by (FK → users)
status ENUM: initiated | in Transit | completed | cancelled
completed_at → auto-set when status = completed
Does NOT auto-update asset status in center_id → transfer must PATCH assets separately

repairs

id (PK UUID), asset_id (FK → assets)
reported_by (FK → users)
repair_center_name, ext_data, expected_return_date
status ENUM: used | in progress | completed
cost, notes

System

notifications_log

id (PK UUID), recipient_mobile, message_type
reference_id → lease pointer, no FK constraint
status ENUM: pending | sent | failed
error_message, sent_at

audit_log

id (PK UUID), user_id (FK → users, SET NULL)
center_id (FK → centers, SET NULL)
action, entity_type, entity_id
old_value, new_value, new_value JSON
new_value always includes actor.mobile + actor.role
ip_address, created_at

Views

sku_stock_by_center (VIEW)

SELECT sku_id, center_id, COUNT(*) AS available_count
FROM assets WHERE status = 'available'
GROUP BY sku_id, center_id
Used by issue context endpoint for availability planning

LAYER 4 — INFRASTRUCTURE & PENDING

Environment Config

DATABASE_URL (MySQL connection string)
CORS_ORIGINS (comma-separated, configurable)
SQLAlchemy pool_pre_ping = True

Auth Model (Current)

No JWT or session tokens
Login: GET /api/v1/users filtered by mobile
Every request: X-Actor-Mobile + X-Actor-Role headers
These headers are req'd reported → backend trusts them completely
OTP valid: exists → implementation PENDING
! No server-side role enforcement

NOT YET IMPLEMENTED (Roadmap)

- RBAC middleware (role enforcement per endpoint)
- Flexible roles & permissions matrix
- SMS / WhatsApp notification dispatcher
- Scheduled jobs: due date alerts, overdue checks
- Repair management API endpoints
- OTP login API endpoints
- Token / lease extension workflow
- UUID format validation on incoming FK fields
- approved, by field population at approval transition
- Single-transaction guarantee for audit writes

Deployment Target TBD

Current: localhost:8080
DB: mysql+pympd driver
Needs: cloud provider decision
File storage for invoices and images
CI/CD pipeline
Hello date residency consideration → Another numbers are stored

LAYER 5 — LEASE REQUEST LIFECYCLE



KEY TRANSITION RULES

Rule 1: Server derives status from asset_id assignments. Frontend cannot force a lease status.

Rule 2: approved, by FK filled at APPROVED transition → captures Approver identity.

Rule 3: Asset status → leased at ISSUED. Asset status → available at RETURNED.

Rule 4: Cross-center returns supported → return_center_id on lease_items may differ from the issuing center.

Rule 5: Extension workflow not yet implemented → PARTIALLY_RETURNED is tracked but no action.